CEE6501 — Lecture 13.2

Force Density Method with Constraints

Learning Objectives

By the end of this lecture, you will be able to:

- explain how the force density method fits within the unified equilibrium update framework
- interpret the force density formulation as a simplified, geometric-only stiffness representation
- show why the linear force density method reduces to a direct solve when $\mathbf{q}$ is prescribed
- explain why introducing constraints leads to a nonlinear problem
- formulate constraint equations in terms of force densities
- outline and implement an iterative solution procedure for constrained form finding
- compare force density with stiffness matrix (SM) and dynamic relaxation (DR) methods

Agenda

- Part 1 — Force density in the unified framework
- Part 2 — Nonlinear force density and constraints
- Part 3 — Deriving the constraint Jacobian
- Part 4 — Practical implementation for force constraints
- Part 5 — Example problems with force constraints
- Part 6 — Introduction to SM and DR methods
- Part 7 — Comparison and computational efficiency

Part 1 - Force Density in the Unified Framework

Reminder of the Unified Framework

From the previous lecture, we wrote a common equilibrium update in the form

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \left(\mathbf{C}^T\mathbf{K}\mathbf{C}\right)^{-1}\left(\mathbf{p} - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}_f\,\mathbf{x}_f - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}\,\mathbf{x}_i\right)$$

Conceptually, this is always the same structure:

$$\text{new coordinates} = \text{current coordinates} + \text{correction from the current residual}$$

The different form-finding methods mainly differ in how the internal branch quantities are defined.

Defining the Terms

- $\mathbf{x}_i$ = current free-node coordinates
- $\mathbf{x}_f$ = fixed-node coordinates (these are fixed and don't change by definition)
- $\mathbf{x}_{i+1}$ = updated free-node coordinates
- $\mathbf{C}$ = free-node branch connectivity matrix
- $\mathbf{C}_f$ = fixed-node branch connectivity matrix
- $\mathbf{p}$ = applied nodal load vector in the coordinate direction being solved
- $\mathbf{L}$ = diagonal matrix of member lengths
- $\mathbf{F}$ = diagonal matrix of member forces
- $\mathbf{K}$ = branch-level stiffness-like quantity used in the update

The residual term is

$$\mathbf{r}_i = \mathbf{p} - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}_f\,\mathbf{x}_f - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}\,\mathbf{x}_i$$

It measures how far the current coordinate state is from equilibrium.

Substitution used in Force Density

In Schek's force density method, the quantity

$$\mathbf{L}^{-1}\mathbf{F}$$

is exactly the diagonal matrix of force densities:

$$\mathbf{Q} = \mathbf{L}^{-1}\mathbf{F}$$

So the unified expression becomes

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \left(\mathbf{C}^T\mathbf{K}\mathbf{C}\right)^{-1}\left(\mathbf{p} - \mathbf{C}^T\mathbf{Q}\mathbf{C}_f\,\mathbf{x}_f - \mathbf{C}^T\mathbf{Q}\mathbf{C}\,\mathbf{x}_i\right)$$

Schek then defines

$$\mathbf{D} = \mathbf{C}^T\mathbf{Q}\mathbf{C} \qquad \text{and} \qquad \mathbf{D}_f = \mathbf{C}^T\mathbf{Q}\mathbf{C}_f$$

which gives

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \left(\mathbf{C}^T\mathbf{K}\mathbf{C}\right)^{-1}\left(\mathbf{p} - \mathbf{D}_f\,\mathbf{x}_f - \mathbf{D}\,\mathbf{x}_i\right)$$

Quick Aside: Where Does the Stiffness Term Come From?

In Schek's force density derivation, a conventional stiffness matrix never appears explicitly.

But within the unified framework, we see the $\mathbf{K}$ term, so some stiffness-like quantity must still be sitting behind the method.

So this raises an important question:

If the unified framework uses $\mathbf{C}^T\mathbf{K}\mathbf{C}$, what is $\mathbf{K}$ for force density?

Recall from Lecture 11 that, for geometrically nonlinear truss analysis, the tangent stiffness had two parts:

$$\mathbf{K}_t = \mathbf{K}_e + \mathbf{K}_g$$

where:

- $\mathbf{K}_e$ is the elastic stiffness term
- $\mathbf{K}_g$ is the geometric stiffness term

$$\mathbf{K}_t = \frac{EA}{L}\begin{bmatrix} c_x^2 & c_x c_y & -c_x^2 & -c_x c_y \\ c_x c_y & c_y^2 & -c_x c_y & -c_y^2 \\ -c_x^2 & -c_x c_y & c_x^2 & c_x c_y \\ -c_x c_y & -c_y^2 & c_x c_y & c_y^2 \end{bmatrix} + \frac{f}{L}\begin{bmatrix} c_y^2 & -c_x c_y & -c_y^2 & c_x c_y \\ -c_x c_y & c_x^2 & c_x c_y & -c_x^2 \\ -c_y^2 & c_x c_y & c_y^2 & -c_x c_y \\ c_x c_y & -c_x^2 & -c_x c_y & c_x^2 \end{bmatrix}$$

From Element-Level Stiffness to Lumped Nodal Stiffness

Instead of full element stiffness matrices, we shift to a **lumped nodal stiffness** view.

You may have seen this idea in structural dynamics, where **mass is lumped at nodes** to simplify the equations of motion.

Here, we do something similar with stiffness.

- each member contributes stiffness directly at the nodes
- full DOF coupling terms are no longer assembled
- the formulation becomes much simpler and more computationally efficient

This is the idea behind **dynamic relaxation**.

Quick Intro Dynamic Relaxation

We still think of stiffness as having two parts, but now these are treated in a reduced scalar sense, rather than as full element matrices.

Instead of a full element matrix, stiffness is approximated as a **diagonal matrix**:

$$\mathbf{K}_{\mathrm{DR}} \approx \begin{bmatrix} k_1 & 0 & 0 & 0 \\ 0 & k_2 & 0 & 0 \\ 0 & 0 & k_3 & 0 \\ 0 & 0 & 0 & k_4 \end{bmatrix}$$

Each diagonal term is the **sum of contributions from connected elements** at each node:

$$k_i = \sum_{e \in i} \left(\frac{EA}{L} + \frac{f}{L} \right)_e$$

Force Density Takes the Simplification One Step Further

dropping the elastic term entirely and retaining only the geometric contribution.

This geometric stiffness is then **lumped at the nodes** and written in scalar form as:

$$\mathbf{K}_{\text{FD}} \approx \begin{bmatrix} k_1 & 0 & 0 & 0 \\ 0 & k_2 & 0 & 0 \\ 0 & 0 & k_3 & 0 \\ 0 & 0 & 0 & k_4 \end{bmatrix}$$

Each diagonal term is the **sum of contributions from connected elements**:

$$k_i = \sum_{e \in i} \left(\frac{f}{L} \right)_e$$

$$k_i = \frac{\text{force}}{\text{length}}$$

This scalar quantity is exactly the **force density**, q

For the Force Density Method, $k = q$

So within the unified framework, the force density stiffness matrix becomes

$$\mathbf{K}_{\text{FD}} = \mathbf{C}^T\mathbf{K}\mathbf{C}$$

Here, the operation $\mathbf{C}^T(\text{diagonal matrix})\mathbf{C}$ plays the same role as before: it collects and sums the member contributions meeting at each node.

Since the member stiffness-like term is now just the **force density**, we obtain

$$\mathbf{C}^T\mathbf{K}\mathbf{C} = \mathbf{C}^T\mathbf{Q}\mathbf{C} = \mathbf{D}$$

So although Schek did not derive the force density method from a stiffness matrix perspective, the unified interpretation shows that the force density method can be understood as using a **simplified, geometric-only, lumped stiffness formulation**.

Continue Substitution used in Force Density

If the branch stiffness-like term is just the force density, then

$$\mathbf{C}^T\mathbf{K}\mathbf{C} = \mathbf{C}^T\mathbf{Q}\mathbf{C} = \mathbf{D}$$

and the unified expression becomes

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{D}^{-1}\left(\mathbf{p} - \mathbf{D}_f\,\mathbf{x}_f - \mathbf{D}\,\mathbf{x}_i\right)$$

Now distribute $\mathbf{D}^{-1}$:

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{D}^{-1}\mathbf{p} - \mathbf{D}^{-1}\mathbf{D}_f\,\mathbf{x}_f - \mathbf{D}^{-1}\mathbf{D}\,\mathbf{x}_i$$

Since $\mathbf{D}^{-1}\mathbf{D} = \mathbf{I}$,

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{D}^{-1}\mathbf{p} - \mathbf{D}^{-1}\mathbf{D}_f\,\mathbf{x}_f - \mathbf{x}_i$$

So the $\mathbf{x}_i$ terms cancel, leaving

$$\mathbf{x}_{i+1} = \mathbf{D}^{-1}\left(\mathbf{p} - \mathbf{D}_f\,\mathbf{x}_f\right)$$

This is exactly the standard **linear force density equation** originally derived by Schek, and matches the form we derived in last week's lecture.

Main Takeaway

When $\mathbf{q}$ is prescribed and remains constant, it does **not** depend on the current geometry (i.e., it is not a function of F or L).

As a result:

- the matrices $\mathbf{D}$ and $\mathbf{D}_f$ are constant
- they do **not** change as the nodal coordinates (x, y, z) update

Because of this, the problem does **not** require iteration.

The force density method reduces to a **single-step linear solve** for the nodal coordinates:

$$\mathbf{x} = \mathbf{D}^{-1} \left(\mathbf{p} - \mathbf{D}_f \, \mathbf{x}_f \right)$$

The same equation is then solved independently for the x, y, and z directions.

Part 2 - Nonlinear Force Density Method

From Prescribed Force Densities to Constrained Form Finding

The linear force density method works because the force densities $\mathbf{q}$ are **given**.

Once $\mathbf{q}$ is prescribed:

- $\mathbf{D}$ and $\mathbf{D}_f$ are constant
- the equilibrium equations are linear
- nodal coordinates are solved in a **single step**

But in many design problems, we do **not** want to prescribe $\mathbf{q}$ directly.

Instead, we impose constraints such as:

- target member forces
- target member lengths
- geometric conditions (distances, symmetry)

Now:

- $\mathbf{q} \in \mathbb{R}^m$ becomes **unknown**
- and must satisfy additional equations

Definition Of The Constraint Function

We define a vector of constraint functions:

$$\mathbf{g}(\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{q}) \in \mathbb{R}^r$$

where:

- $\mathbf{g}$ = vector of constraint residuals
- r = number of constraints

Each individual constraint is written as

$$g_k(\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{q}) = 0$$

where g_k measures the difference between the **target constraint value** and the **current value**

For example, $g = L_{\text{actual}} - L_{\text{target}}$

Interpretation

- $g_k = 0 \rightarrow$ constraint is satisfied
- $g_k \neq 0 \rightarrow$ constraint is not satisfied

Where the Nonlinearity Comes From

The nodal coordinates depend on the force densities:

$$\mathbf{x} = \mathbf{x}(\mathbf{q}), \quad \mathbf{y} = \mathbf{y}(\mathbf{q}), \quad \mathbf{z} = \mathbf{z}(\mathbf{q})$$

Substitute into the constraint function:

$$\mathbf{g}^*(\mathbf{q}) = \mathbf{g}(\mathbf{x}(\mathbf{q}), \mathbf{y}(\mathbf{q}), \mathbf{z}(\mathbf{q}), \mathbf{q})$$

Constraints depend on $\mathbf{q}$ both directly and through geometry = Nonlinear problem in $\mathbf{q}$

Example Constraint — Target Member Force

Prescribe:

$$F_i = F_i^{\text{target}}$$

Using force density:

$$F_i = q_i L_i$$

Constraint function:

$$g_i(\mathbf{q}) = q_i L_i(\mathbf{q}) - F_i^{\text{target}} = 0$$

Nonlinear because L_i depends on $\mathbf{q}$

Example Constraint — Target Member Length

Prescribe:

$$L_i = L_i^{\text{target}}$$

Constraint function:

$$g_i(\mathbf{q}) = L_i(\mathbf{q}) - L_i^{\text{target}} = 0$$

Again nonlinear because length depends on $\mathbf{q}$

These additional equations, which are generally nonlinear, will not in general be satisfied by the initial force density vector $\mathbf{q}^{(0)}$

where $\mathbf{q}^{(0)}$ corresponds to the shape obtained from the initial linear force density form-finding step.

The goal is therefore to find an updated force density vector of the form

$$\mathbf{q}^{(1)} = \mathbf{q}^{(0)} + \Delta\mathbf{q}$$

such that

$$\mathbf{g}^{*}(\mathbf{q}^{(1)}) = \mathbf{0}$$

Linearizing About the Current State

Because the constraint equations are nonlinear, we do **not** solve

$$\mathbf{g}^*(\mathbf{q}^{(1)}) = \mathbf{0}$$

directly.

Instead, we start from the current estimate $\mathbf{q}^{(0)}$ and solve for an increment $\Delta\mathbf{q}$.

Using a first-order linearization about $\mathbf{q}^{(0)}$, we write

$$\mathbf{g}^*(\mathbf{q}^{(0)}) + \frac{\partial \mathbf{g}^*(\mathbf{q}^{(0)})}{\partial \mathbf{q}}\,\Delta\mathbf{q} = \mathbf{0}$$

So, rather than solving the full nonlinear constraint equations at once, we solve a **local linear approximation** for the correction $\Delta\mathbf{q}$.

Writing the Linearized System

For simplicity, Schek defines

$$\mathbf{G}^t = \frac{\partial \mathbf{g}^*(\mathbf{q}^{(0)})}{\partial \mathbf{q}} \in \mathbb{R}^{r \times m}$$

$$\mathbf{r} = -\mathbf{g}^*(\mathbf{q}^{(0)}) \in \mathbb{R}^{r \times 1}$$

where:

- m = number of members, or unknown force densities
- r = number of constraint equations

The linearized system can then be written as

$$\mathbf{G}^t \, \Delta\mathbf{q} = \mathbf{r}$$

This system is solved for the force density correction $\Delta\mathbf{q}$.

The Jacobian Matrix, $\mathbf{G}^t$

Here, the superscript t does **not** mean transpose.

It is simply Schek's notation for the **tangent**, or linearized, constraint matrix.

$$\mathbf{G}^t = \frac{\partial \mathbf{g}^*(\mathbf{q}^{(0)})}{\partial \mathbf{q}} \in \mathbb{R}^{r \times m}$$

Each entry of $\mathbf{G}^t$ is a partial derivative:

$$G^t_{kj} = \frac{\partial g^*_k}{\partial q_j}$$

So:

- each **row** corresponds to one constraint equation
- each **column** corresponds to one force density variable

For example, the Jacobian has the form

$$\mathbf{G}^t = \begin{bmatrix} \frac{\partial g_1^*}{\partial q_1} & \frac{\partial g_1^*}{\partial q_2} & \cdots & \frac{\partial g_1^*}{\partial q_m} \\ \frac{\partial g_2^*}{\partial q_1} & \frac{\partial g_2^*}{\partial q_2} & \cdots & \frac{\partial g_2^*}{\partial q_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial g_r^*}{\partial q_1} & \frac{\partial g_r^*}{\partial q_2} & \cdots & \frac{\partial g_r^*}{\partial q_m} \end{bmatrix}$$

This matrix tells us how sensitive each constraint is to changes in each force density.

Recall Geometrically Nonlinear Analysis

This linearization step is directly analogous to the **Newton–Raphson method** used for geometrically nonlinear analysis Recall from Lecture 11–2, we wrote the update as:

$$\mathbf{K}_{ff,t}^{(i)}\,\Delta\mathbf{u}_f^{(i)} = \mathbf{r}^{(i)}$$

$$\Delta\mathbf{u}_f^{(i)} = \mathbf{K}_{ff,t}^{-1(i)}\,\mathbf{r}^{(i)}$$

where the **tangent stiffness matrix** is defined as

$$\mathbf{K}_t = \left[\frac{\partial f_i}{\partial u_j}\right] \qquad i,j = 1,2,3,4$$

where:

- $\mathbf{K}_t$ = linearization of internal forces at the current state
- $\mathbf{r}$ = imbalance (residual) at the current state
- $\Delta\mathbf{u}$ = correction to the displacement

Analogy to Force Density Constraints

- $\mathbf{G}^t = \dfrac{\partial \mathbf{g}^*}{\partial \mathbf{q}}$
  → tangent matrix of the constraints
- $\mathbf{r} = -\mathbf{g}^*(\mathbf{q}^{(0)})$
  → constraint residual
- $\boldsymbol{\Delta}\mathbf{q}$
  → correction to the force densities

We are solving a nonlinear problem by:

- linearizing about the current point
- solving a **local linear system**
- updating the unknowns
- repeating until convergence

Same setup as geometric nonlinear analysis, just with **force densities instead of displacements**

Once linearlized you solve for the error correct term and then can calculate updated force densities

Update:

$$\mathbf{q}^{(i+1)} = \mathbf{q}^{(i)} + \Delta\mathbf{q}$$

Update $\mathbf{G}^t$ based on $\mathbf{q}^{(i+1)}$

Underdetermined System

In most practical and solvable cases, the number of additional constraints is smaller than the number of members.

Here:

- m = number of members, or equivalently the number of unknown force densities
- r = number of additional constraints

so typically,

$$r < m$$

This means there are fewer equations than unknowns.

As a result, the linearized system does **not** have a unique solution.

Instead, there are multiple possible solutions for $\mathbf{\Delta q}$.

Selecting a Solution

Since the system is underdetermined, there are infinitely many possible solutions for $\Delta\mathbf{q}$.

In fact, the solution space has

$$m - r$$

linearly independent directions.

So we need an additional criterion to select a **unique and meaningful** solution.

Minimum-Norm Solution

Schek chooses the solution that minimizes the total change in the force densities:

$$\Delta\mathbf{q}^T\Delta\mathbf{q} \;\rightarrow\; \min$$

Let

$$\Delta\mathbf{q} = \begin{bmatrix} \Delta q_1 \\ \Delta q_2 \\ \vdots \\ \Delta q_m \end{bmatrix}$$

Then:

$$\Delta\mathbf{q}^T\Delta\mathbf{q} = \Delta q_1^2 + \Delta q_2^2 + \cdots + \Delta q_m^2$$

So this is simply the **sum of squares** of all changes in the force densities:

$$\Delta\mathbf{q}^T\Delta\mathbf{q} = \|\Delta\mathbf{q}\|^2$$

Interpretation

This selects the update that is **closest to the current force density state**.

So, among all possible solutions, we choose the one that:

- makes the **smallest overall change** in $\mathbf{q}$
- gives a **smooth and stable update**
- avoids unnecessarily large changes in individual members

Implementing this is covered in more detail in Section 3.1 of the Schek paper

Iterative Procedure

1. solve the equilibrium problem for the current force densities to obtain the geometry

$$\mathbf{x}(\mathbf{q}),\ \mathbf{y}(\mathbf{q}),\ \mathbf{z}(\mathbf{q})$$

2. evaluate the constraint residuals, $\mathbf{g}^*(\mathbf{q})$

3. form the linearized constraint matrix, $\mathbf{G}^t$

4. solve for the force density correction

$$\mathbf{G}^t \Delta\mathbf{q} = \mathbf{r}$$

5. update the force densities

$$\mathbf{q}^{(i+1)} = \mathbf{q}^{(i)} + \Delta\mathbf{q}$$

6. check convergence and repeat if needed

Part 3 - Deriving the Constraints Jacobian Matrix

The Goal

We want to solve the linearized constraint system

$$\mathbf{G}^t \, \Delta\mathbf{q} = \mathbf{r}$$

So the key question becomes:

How do we actually compute $\mathbf{G}^t$?

Recall that

$$\mathbf{G}^t = \frac{\partial \mathbf{g}^*}{\partial \mathbf{q}}$$

where:

- $\mathbf{q}$ = vector of force densities
- $\mathbf{g}^*(\mathbf{q})$ = vector of constraint residuals after the coordinates have been written as functions of $\mathbf{q}$

The difficulty is that the constraints depend on $\mathbf{q}$ in two ways:

- **directly** through $\mathbf{q}$ itself
- **indirectly** through the coordinates $\mathbf{x}(\mathbf{q})$, $\mathbf{y}(\mathbf{q})$, and $\mathbf{z}(\mathbf{q})$

Using the Chain Rule

Schek writes the Jacobian of the constraint equations using the chain rule:

$$\mathbf{G}^t = \frac{\partial \mathbf{g}^*}{\partial \mathbf{q}} = \frac{\partial \mathbf{g}}{\partial \mathbf{x}}\frac{\partial \mathbf{x}}{\partial \mathbf{q}} + \frac{\partial \mathbf{g}}{\partial \mathbf{y}}\frac{\partial \mathbf{y}}{\partial \mathbf{q}} + \frac{\partial \mathbf{g}}{\partial \mathbf{z}}\frac{\partial \mathbf{z}}{\partial \mathbf{q}} + \frac{\partial \mathbf{g}}{\partial \mathbf{q}}$$

This equation says:

- first measure how the constraints change with the coordinates
- then measure how the coordinates change with the force densities
- and also include any direct dependence of the constraints on $\mathbf{q}$

So the main bottleneck is computing

$$\frac{\partial \mathbf{x}}{\partial \mathbf{q}}, \qquad \frac{\partial \mathbf{y}}{\partial \mathbf{q}}, \qquad \frac{\partial \mathbf{z}}{\partial \mathbf{q}}$$

Additional Content Provided

See the extra notes for more detail on the Jacobian derivation and implementation.

This material also shows how to formulate constraint equations for several common cases:

1. node distance constraints (strained length)
2. member length constraints (unstrained length)
3. member force constraints
4. mixed constraint cases

Part 4 - Simple Implementation for Member Force Constraints

A Simpler Implementation Path

Rather than implementing the full constrained solver from Schek immediately, a very practical first step is to impose **member force targets** directly.

This can be done using the unified formulation:

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \left(\mathbf{C}^T\mathbf{K}\mathbf{C}\right)^{-1}\left(\mathbf{p} - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}_f\,\mathbf{x}_f - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}\,\mathbf{x}_i\right)$$

What Changes Compared to Linear Implementation ?

In the standard force density method:

- $\mathbf{Q}$ is **given and constant**
- the problem is **linear** → single-step solve

In the force constrained formulation:

- $\mathbf{F}$ = target member forces (fixed)
- $\mathbf{L}$ = member lengths (depend on nodal positions, current geometry of the structure)
- $\mathbf{Q} = \mathbf{L}^{-1}\mathbf{F}$ = force densities (changing, since lengths are changing)

$$\mathbf{Q}(\mathbf{x}_i) = \mathbf{L}(\mathbf{x}_i)^{-1}\mathbf{F}$$

Why This Becomes Nonlinear

Because:

- lengths, $\mathbf{L}(\mathbf{x}_i)$, depend on the nodal coordinates
- nodal coordinates are what we are solving for

So:

$$\mathbf{Q}_i = \mathbf{Q}(\mathbf{x}_i)$$

The stiffness matrix now depends on the current geometry:

$$\mathbf{K} = \mathbf{Q}(\mathbf{x}_i) = \mathbf{L}(\mathbf{x}_i)^{-1}\mathbf{F}$$

and therefore

$$(\mathbf{C}^T\mathbf{K}\mathbf{C})^{-1} = (\mathbf{C}^T\mathbf{Q}(\mathbf{x}_i)\mathbf{C})^{-1}$$

Main Idea

In the linear force density method:

- $\mathbf{Q}$ is prescribed and stays fixed
- there is no direct control over the final member forces
- but the system is **linear**

$$\text{fixed } \mathbf{Q} \quad \Rightarrow \quad \text{linear problem}$$

In the force-controlled version:

- $\mathbf{F}$ is prescribed and stays fixed
- But the force densities are updated from the current geometry

$$\mathbf{Q}(\mathbf{x}_i) = \mathbf{L}(\mathbf{x_i})^{-1}\mathbf{F}$$

- so the system matrix changes as the nodal positions change
- iteration is required until the geometry, force densities, and residual all converge

$$\mathbf{Q}(\mathbf{x}_i) \quad \Rightarrow \quad \text{nonlinear problem}$$

Algorithm Summary

The overall algorithm structure does **not** actually change.

The difference is that, in the linear force density method convergence is reached in a **single step**.

In the nonlinear force density method the same sequence of steps must be repeated until convergence.

Main difference:

- **linear force density:** $\mathbf{Q}$ is fixed, go through loop 1 time only.
- **force-constrained force density:** $\mathbf{Q}$ changes with geometry, more than 1 loop required.

1. choose an initial geometry and form the connectivity matrices, $\mathbf{C}, \mathbf{C}_f$. These depend only on the network topology, so they do **not** change during the iteration.

2. prescribe the target member forces, $\mathbf{F}$

3. compute the initial member lengths from the starting geometry, $\mathbf{L}_i = \mathbf{L}(\mathbf{x}_i)$

4. compute the current force densities from, $\mathbf{Q}_i = \mathbf{L}_i^{-1}\mathbf{F}$

5. assemble the force density matrices

$$\mathbf{D}_i = \mathbf{C}^T\mathbf{Q}_i\mathbf{C} \qquad \text{and} \qquad \mathbf{D}_{f,i} = \mathbf{C}^T\mathbf{Q}_i\mathbf{C}_f$$

6. solve the linear force density step for the updated free-node coordinates

$$\mathbf{x}_{i+1} = \mathbf{D}_i^{-1}\left(\mathbf{p} - \mathbf{D}_{f,i}\,\mathbf{x}_f\right)$$

7. update the nodal positions and recompute the member lengths, $\mathbf{L}_{i+1} = \mathbf{L}(\mathbf{x}_{i+1})$, from the new geometry

8. check convergence using a chosen criterion (e.g., change in nodal position, change in member length, change in total length, or residual norm)

9. if not converged, repeat steps 4 through 8

Convergence Criterion

A simple convergence measure is the relative change in total member length (as a proxy for force density):

$$\text{error} = \frac{\left| L_{\text{tot}}^{(i+1)} - L_{\text{tot}}^{(i)} \right|}{L_{\text{tot}}^{(i)}}$$

but other convergence measures are possible as well.

Part 5 - Force Density Examples with Force Constraints

Repository Link

https://github.com/Bruun-Automation-Research-Lab/matrix-form-finding

Running Code

Run from the `./formfinder/main.py` file

Input files specified in `./formfinder/structures` folder

Text data saved to `./debug` folder

Output plots and daya saved to Text data saved to `./output` folder

Running from the `main.py` File

To run the iterative force density solver, open `main.py` and set:

- `solver=solvers[2]` for `FD_nonlinear`
- `structure=structs[...]` to whichever example you want to run

```
if __name__ == "__main__":
    solvers = {
        1: "FD_linear",
        2: "FD_nonlinear",
        3: "SM",
        4: "DR_imp",
        5: "DR_leap",
    }
    structs = {
        1: "Schek_2D",
        2: "Schek_Modified",
        3: "Veenendaal",
        4: "Pauletti",
        5: "CEE6501_2D",
        6: "CEE6501_PointLoad",
        7: "Star",
    }
    run = FormFinder(
        solver=solvers[2], structure=structs[3], debug=True, plot_save=True
    )
    run.solve()
```

Structure 1 — Schek Structure With An Extra Node

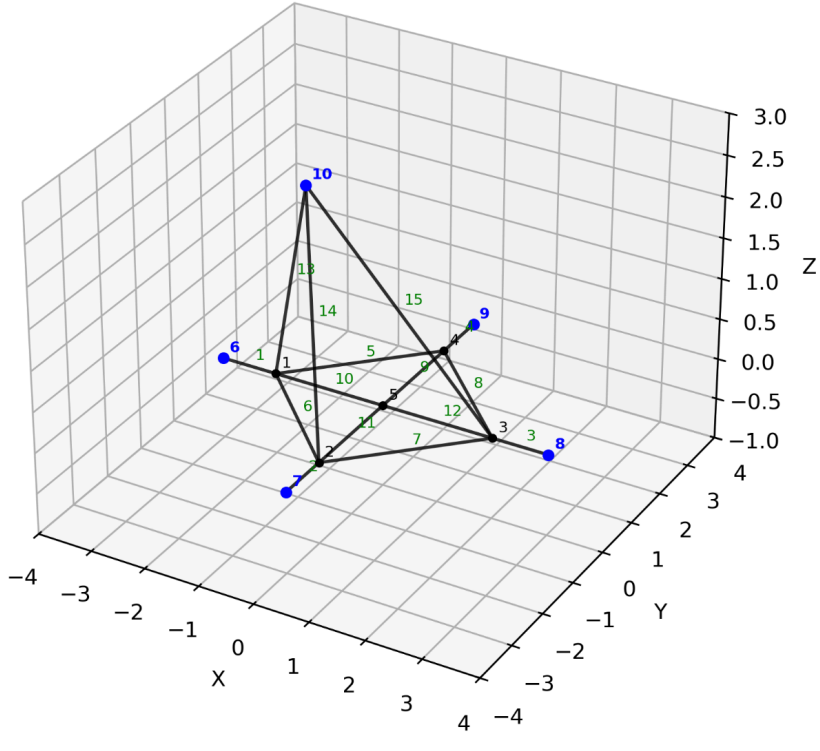
Starting from the planar Schek structure, add one additional node out of plane in the z-direction and connect it to the existing structure with three new members.

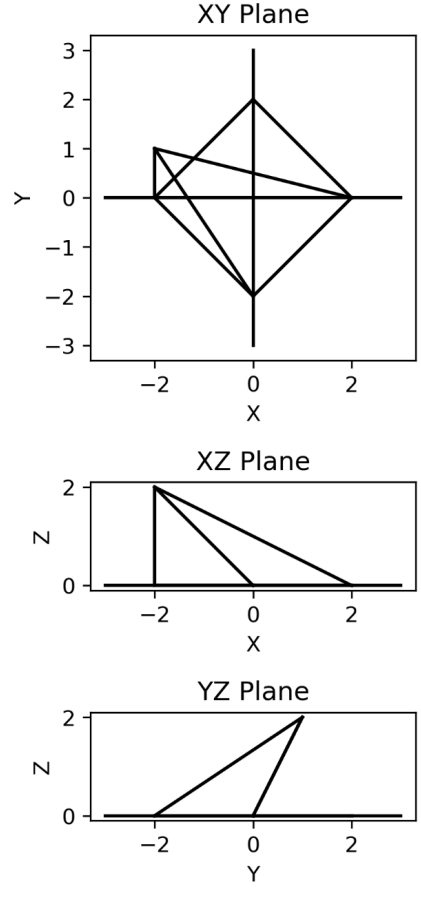
Set the target internal member forces to

$$\mathbf{f} = [3,\ 3,\ 3,\ 3,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1]^T$$

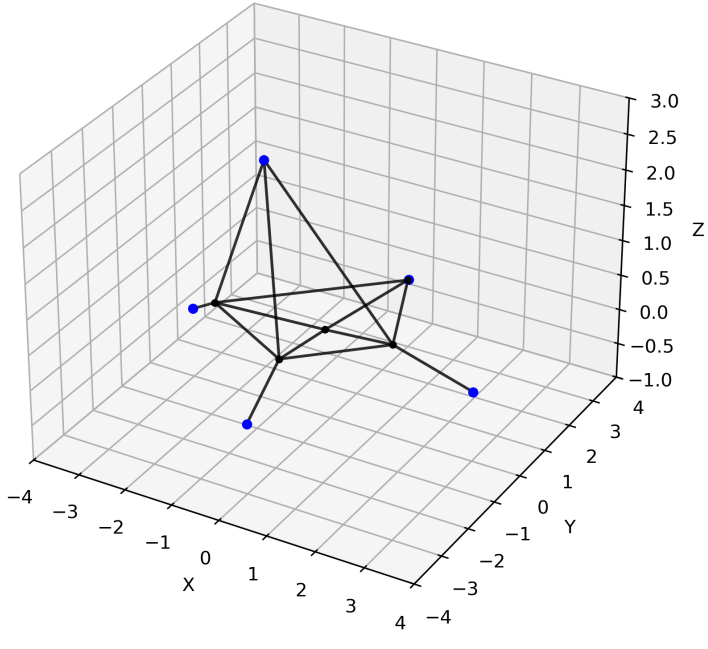
That is, the first four members are assigned a target force of 3, and all remaining members are assigned a target force of 1.

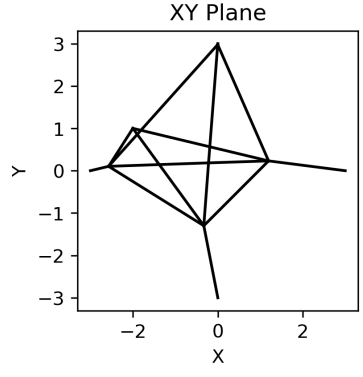
Description of Structure

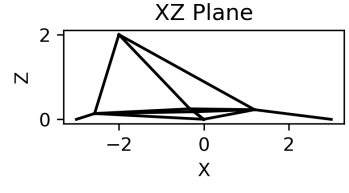


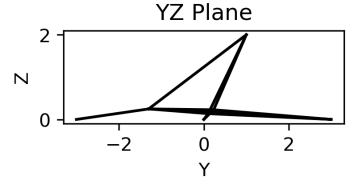


Results (Start -> Final)

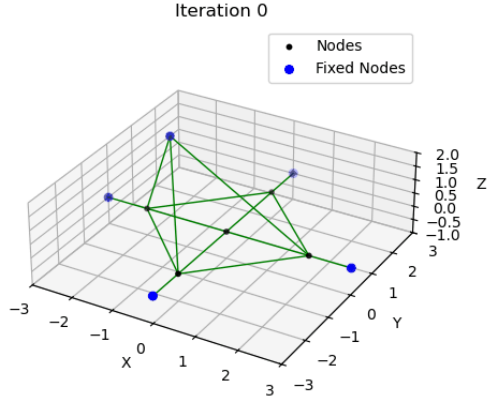








Animation (Only Visible In Jupyter Notebook)




```
Iteration 000: Total Len = 331.139, Max error = 1.486e+01
Iteration 001: Total Len = 330.595, Max error = 5.437e-01
Iteration 002: Total Len = 330.360, Max error = 2.350e-01
Iteration 003: Total Len = 330.228, Max error = 1.325e-01
Iteration 004: Total Len = 330.145, Max error = 8.218e-02
Iteration 005: Total Len = 330.093, Max error = 5.236e-02
Iteration 006: Total Len = 330.059, Max error = 3.355e-02
Iteration 007: Total Len = 330.038, Max error = 2.150e-02
Iteration 008: Total Len = 330.024, Max error = 1.375e-02
Iteration 009: Total Len = 330.015, Max error = 8.785e-03
Iteration 010: Total Len = 330.010, Max error = 5.603e-03
Iteration 011: Total Len = 330.006, Max error = 3.569e-03
Iteration 012: Total Len = 330.004, Max error = 2.272e-03
Iteration 013: Total Len = 330.003, Max error = 1.445e-03
Iteration 014: Total Len = 330.002, Max error = 9.180e-04
Convergence achieved! Performing one final update.
Final update complete. Exiting loop.
```

Final Node Positions and Branch Forces & Lengths

```
#----------------------------------------
FINAL NODES
[n x 3]
```

	x	y	z
	1 x	1 x	1 x
1	-2.572	0.103	0.138
2	-0.330	-1.299	0.249
3	1.195	0.232	0.228
4	-0.003	2.978	0.001
5	-0.219	0.185	0.179
6	-3.000	0.000	0.000
7	0.000	-3.000	0.000
8	3.000	0.000	0.000
9	0.000	3.000	0.000
10	-2.000	1.000	2.000

```
#----------------------------------------
FINAL FORCE/LENGTH/FORCE DENSITY
[m x 1]
```

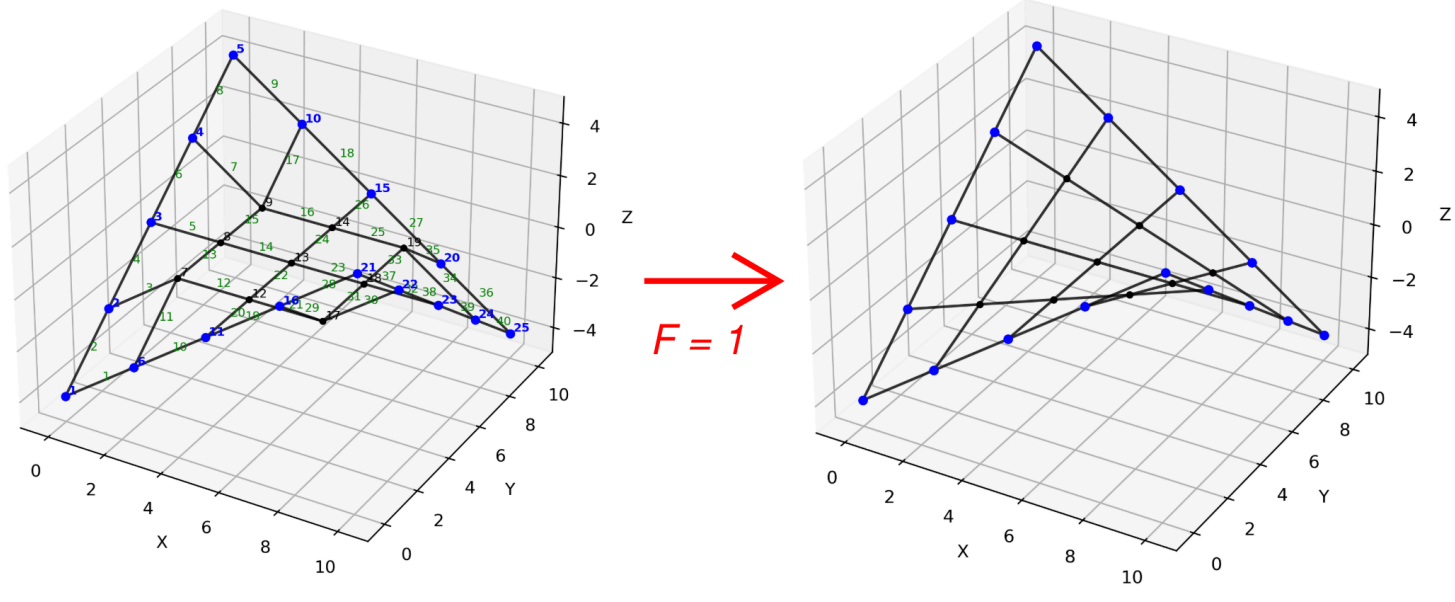
	Force	Length	q
	1 x	1 x	1 x
1	3.0000	0.4617	6.4976
2	3.0000	1.7507	1.7136
3	3.0000	1.8344	1.6355
4	3.0000	0.0227	132.0626
5	1.0000	3.8572	0.2593
6	1.0000	2.6459	0.3779
7	1.0000	2.1606	0.4628
8	1.0000	3.0042	0.3329
9	1.0000	2.8066	0.3563
10	1.0000	2.3547	0.4247
11	1.0000	1.4894	0.6714
12	1.0000	1.4149	0.7068
13	1.0000	2.1446	0.4663
14	1.0000	3.3376	0.2996
15	1.0000	3.7331	0.2679

Structure 2 — Veenendaal Validation Structure

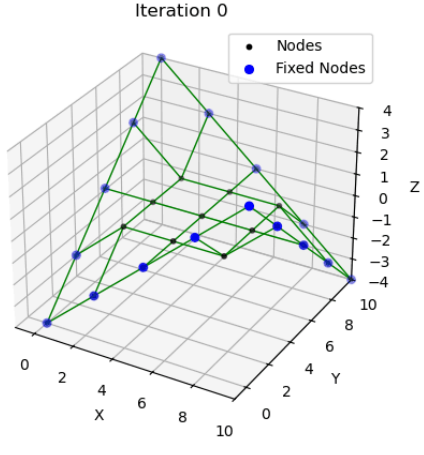
A hyperbolic paraboloid cable net with fixed boundary nodes is used as a validation example to compare all three form-finding methods.

For this case, the target internal member force is taken as uniform in every member: $f = 1$

Results (Start -> Final)



Animation (Only Visible In Jupyter Notebook)




```
Iteration 000: Total Len = 331.139, Max error = 1.486e+01
Iteration 001: Total Len = 330.595, Max error = 5.437e-01
Iteration 002: Total Len = 330.360, Max error = 2.350e-01
Iteration 003: Total Len = 330.228, Max error = 1.325e-01
Iteration 004: Total Len = 330.145, Max error = 8.218e-02
Iteration 005: Total Len = 330.093, Max error = 5.236e-02
Iteration 006: Total Len = 330.059, Max error = 3.355e-02
Iteration 007: Total Len = 330.038, Max error = 2.150e-02
Iteration 008: Total Len = 330.024, Max error = 1.375e-02
Iteration 009: Total Len = 330.015, Max error = 8.785e-03
Iteration 010: Total Len = 330.010, Max error = 5.603e-03
Iteration 011: Total Len = 330.006, Max error = 3.569e-03
Iteration 012: Total Len = 330.004, Max error = 2.272e-03
Iteration 013: Total Len = 330.003, Max error = 1.445e-03
Iteration 014: Total Len = 330.002, Max error = 9.180e-04
Convergence achieved! Performing one final update.
Final update complete. Exiting loop.
```

Comparison with Constant Force-Density Solution

CONSTANT FORCE-DENSITY

#--
FINAL NODES
[n x 3]

	x	y	z
	1 x	1 x	1 x
1	0.000	0.000	-4.000
2	0.000	2.500	-2.000
3	0.000	5.000	0.000
4	0.000	7.500	2.000
5	0.000	10.000	4.000
6	2.500	0.000	-2.000
7	2.500	2.500	-1.000
8	2.500	5.000	0.000
9	2.500	7.500	1.000
10	2.500	10.000	2.000
11	5.000	0.000	0.000
12	5.000	2.500	0.000
13	5.000	5.000	0.000
14	5.000	7.500	0.000
15	5.000	10.000	0.000
16	7.500	0.000	2.000
17	7.500	2.500	1.000
18	7.500	5.000	0.000
19	7.500	7.500	-1.000
20	7.500	10.000	-2.000
21	10.000	0.000	4.000
22	10.000	2.500	2.000
23	10.000	5.000	0.000
24	10.000	7.500	-2.000
25	10.000	10.000	-4.000

CONSTANT FORCE

#--
FINAL NODES
[n x 3]

	x	y	z
	1 x	1 x	1 x
1	0.000	0.000	-4.000
2	0.000	2.500	-2.000
3	0.000	5.000	0.000
4	0.000	7.500	2.000
5	0.000	10.000	4.000
6	2.500	0.000	-2.000
7	2.506	2.506	-0.997
8	2.508	5.000	0.000
9	2.506	7.494	0.997
10	2.500	10.000	2.000
11	5.000	0.000	0.000
12	5.000	2.508	-0.000
13	5.000	5.000	0.000
14	5.000	7.492	0.000
15	5.000	10.000	0.000
16	7.500	0.000	2.000
17	7.494	2.506	0.997
18	7.492	5.000	0.000
19	7.494	7.494	-0.997
20	7.500	10.000	-2.000
21	10.000	0.000	4.000
22	10.000	2.500	2.000
23	10.000	5.000	0.000
24	10.000	7.500	-2.000
25	10.000	10.000	-4.000

Structure 2 — Pauletti Validation Structure

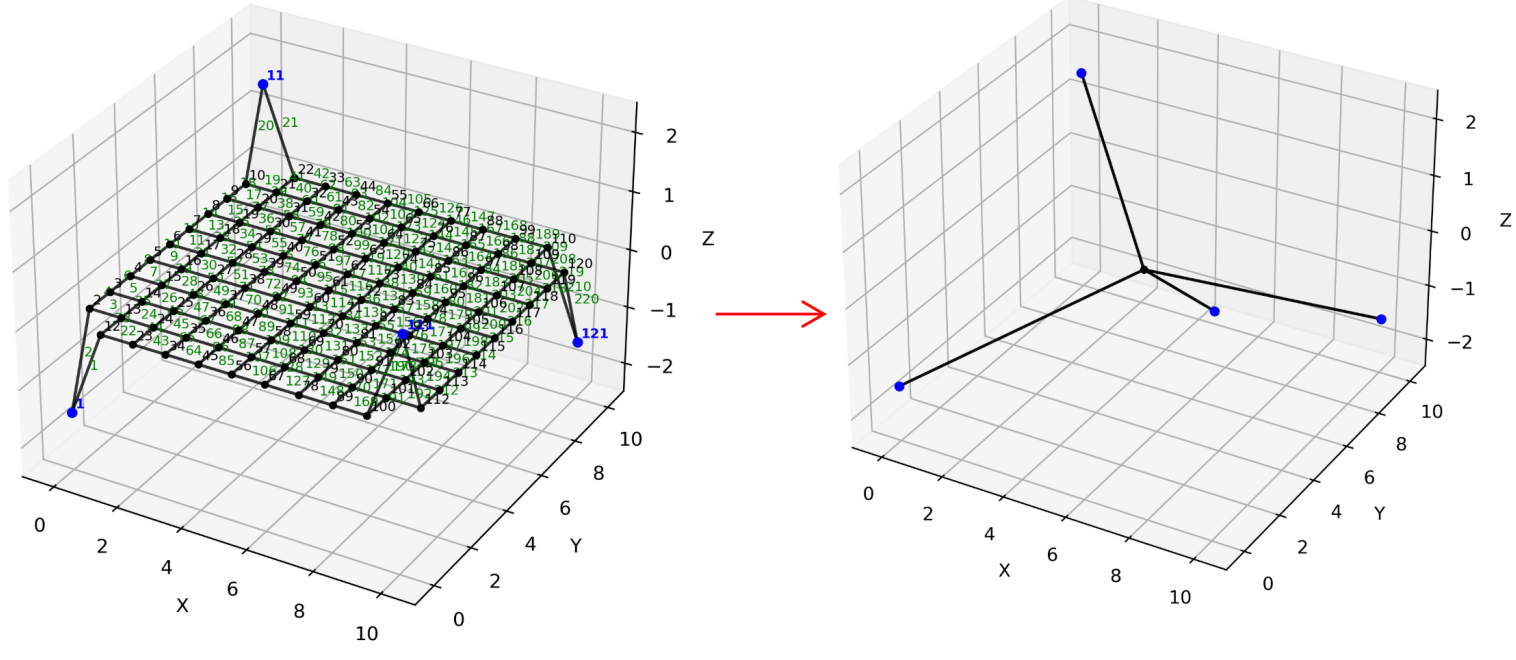
A prescribed **force state** is not guaranteed to produce a nontrivial equilibrium shape in the standard force density method.

Unlike prescribed **force densities**, prescribed member forces can overconstrain the problem, since the required force densities depend on the unknown final lengths.

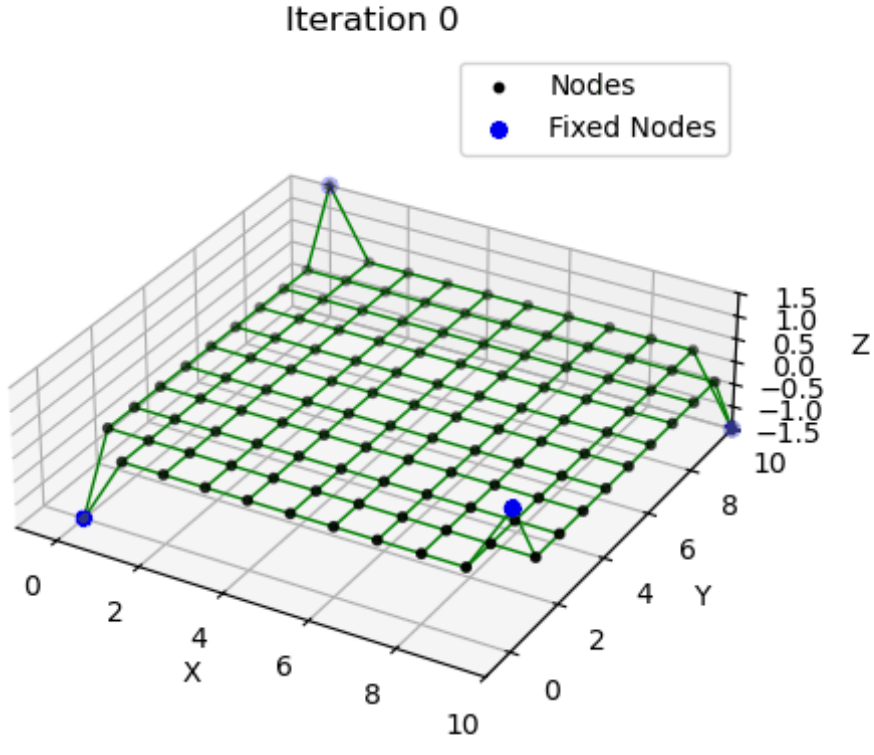
As a result, the solution may fail to converge, or may converge to a trivial or degenerate configuration.

For the Pauletti validation structure, this issue appears when enforcing a highly constrained target force ratio, for example: $f_{\text{outer}} : f_{\text{inner}} = 5 : 1$

Results (Start -> Final)



Animation (Only Visible in Jupyter Notebook)



What If We Don't Want To Prescribe Every Force?

The previous example shows that prescribing a full **force state** can lead to trivial or non-convergent solutions.

A practical workaround is to **mix formulations**:

- prescribe forces in a subset of members
- prescribe force densities in the rest

This relaxes the problem and can restore convergence.

Limitation of Force Density

The force density method requires prescribing an internal force state:

- either $\mathbf{q}$ (force densities)
- or $\mathbf{f}$ (member forces)

This is both its strength and its limitation.

- In the standard **linear** force density method, prescribing $\mathbf{q}$ leads directly to an equilibrium shape
- But not every prescribed **force state** is compatible with a valid nontrivial equilibrium
- In those cases, the solution may fail to converge, or may converge to a trivial or degenerate configuration

Natural Question

What if we do not want to prescribe the internal force state in advance?

- no target member forces
- no target force densities
- just let the structure find an equilibrium shape

Part 6 — Stiffness Matrix (SM) And Dynamic Relaxation (DR) Methods

Motivation

If we do not want to prescribe the internal force state at all, and instead want it to develop as part of the solution. This can be done with:

- Stiffness Matrix (SM) methods
- Dynamic Relaxation (DR)

Unified Form

Both SM and DR can be interpreted within the same general framework:

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \left(\mathbf{C}^T\mathbf{K}\mathbf{C}\right)^{-1}\left(\mathbf{p} - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}_f\,\mathbf{x}_f - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}\,\mathbf{x}_i\right)$$

The overall structure of the update is the same, but the meaning of $\mathbf{K}$ and $\mathbf{F}$ changes from method to method.

The key difference is:

- **FD** prescribes the internal force state
- **SM** and **DR** compute the internal force state from deformation and equilibrium

Stiffness Matrix, $\mathbf{K}$, For Each Method

The three methods mainly differ in how they represent stiffness.

- **Force Density (FD)**
  - keeps only the geometric contribution
  - each member contributes a scalar force-density term
  - these are summed at the node through the $\mathbf{C}^T(\cdot)\mathbf{C}$ operation

$$k_i = \sum_{e \in i} \left(\frac{f}{L} \right) = \sum_{e \in i} q_e$$

- **Dynamic Relaxation (DR)**
  - uses a lumped nodal stiffness
  - keeps both elastic and geometric contributions
  - these are summed at the node through the $\mathbf{C}^T(\cdot)\mathbf{C}$ operation

$$k_i = \sum_{e \in i} \left(\frac{EA}{L} + \frac{f}{L} \right)_e$$

- **Stiffness Matrix (SM)**
    - keeps a full nodal stiffness contribution
    - each member is represented with a 3×3 **matrix** nodal stiffness matrix
    - these are summed at the node through the $\mathbf{C}^T(\cdot)\mathbf{C}$ operation

$$\mathbf{K}_i = \sum_{e \in i} \left(\mathbf{K}_e^{(e)} + \mathbf{K}_e^{(g)} \right)$$

So compared to FD and DR, SM does not reduce stiffness to a single scalar at the node. It still works from a nodal point of view, but now the x, y, and z directions at each node remain coupled.

Element Stiffness Matrix (3 × 3 Form)

For the stiffness matrix method, the stiffness contribution at a node is written as the sum of an elastic and a geometric part:

$$\mathbf{K}_e = \mathbf{K}_e^{(e)} + \mathbf{K}_e^{(g)}$$

Elastic stiffness:

$$\mathbf{K}_e^{(e)} = \frac{EA}{L_0}\mathbf{G}^T\mathbf{G}$$

Geometric stiffness:

$$\mathbf{K}_e^{(g)} = \frac{F}{L}\left(\mathbf{I}_3 - \mathbf{G}^T\mathbf{G}\right)$$

where:

- $\mathbf{G} = [c_x \; c_y \; c_z]$ = direction cosine vector
- $\mathbf{I}_3$ = 3×3 identity matrix
- EA = axial stiffness
- L_0 = initial length
- L = current length
- F = current axial force

Interpretation

- $\mathbf{G}^T\mathbf{G}$ projects stiffness in the member direction
- $\mathbf{I}_3 - \mathbf{G}^T\mathbf{G}$ gives the geometric contribution associated with the current force state
- the elastic term captures material resistance to stretching
- the geometric term captures how the current axial force changes the effective stiffness

This is closely related to the geometrically nonlinear truss formulation from earlier in the course.

The key difference is that here we are looking from the perspective of the node, as a 3×3 **matrix**, rather than the full element-level start-node/end-node coupling, written as a 6×6 matrix.

Force Term, F, For Each Method

- **Force Density (FD)**, force follows directly from prescribed force density:

$$f = qL$$

- **Dynamic Relaxation (DR)** and **Stiffness Matrix (SM)**, force is not prescribed directly, instead it develops from the current deformation of the member:

$$f = \frac{EA}{L_0}(L - L_0) + F_0$$

where:

- L = current member length
- L_0 = initial or unstressed length
- F_0 = initial force, such as prestress or pretension

So in SM and DR, the internal forces evolve during the solution as the geometry changes.

Comparison of FD, DR, And SM

Quantity	Force Density (FD)	Dynamic Relaxation (DR)	Stiffness Matrix (SM)
Main Idea	prescribe force densities and solve for shape	iteratively relax to equilibrium	solve equilibrium through stiffness equations
What Is Prescribed	$\mathbf{q}$	geometry, EA, loads, supports	geometry, EA, loads, supports
Internal Force State	prescribed indirectly through $\mathbf{q}$	develops during iteration	develops during iteration
Stiffness Representation	lumped geometric only	lumped elastic + geometric	full elastic + geometric
Member Stiffness Form	scalar	scalar	3×3 matrix
K-Matrix Size	$m \times m$	$m \times m$	$3m \times 3m$
Coordinate Coupling	none	none	full coupling of x, y, z
Coordinates Solved	x, y, z separately	x, y, z separately	all together in one vector

General Implementation Notes

To keep the structure of the methods parallel, it helps to think of all three in terms of:

1. define stiffness-like quantity
2. define internal force term
3. assemble equilibrium system
4. solve for updated geometry

Force Density (FD)

- input: prescribed $\mathbf{q}$
- build diagonal matrix $\mathbf{Q}$ of size $m \times m$
- form: $\mathbf{D}$ and $\mathbf{D}_f$
- solve x, y, and z separately
- very efficient because the problem is linear once $\mathbf{q}$ is known

Dynamic Relaxation (DR)

- input: geometry, stiffness, loads, damping
- compute lumped scalar nodal stiffness terms
- global stiffness-like matrix is diagonal, $m \times m$
- solve x, y, and z separately by explicit iteration in pseudo-time
- update coordinates repeatedly until residual forces approach zero
- damping is typically needed to avoid persistent oscillation and help convergence

Stiffness Matrix (SM)

- input: geometry, stiffness, loads
- compute full 3×3 nodal contributions from each member
- assemble a global matrix of size $3m \times 3m$
- solve all DOFs simultaneously using a stacked displacement vector

$$\mathbf{u} = [x_1, y_1, z_1, x_2, y_2, z_2, \ldots]^T$$

- update coordinates repeatedly until residual forces approach zero

Part 7 - Example of DR and SM

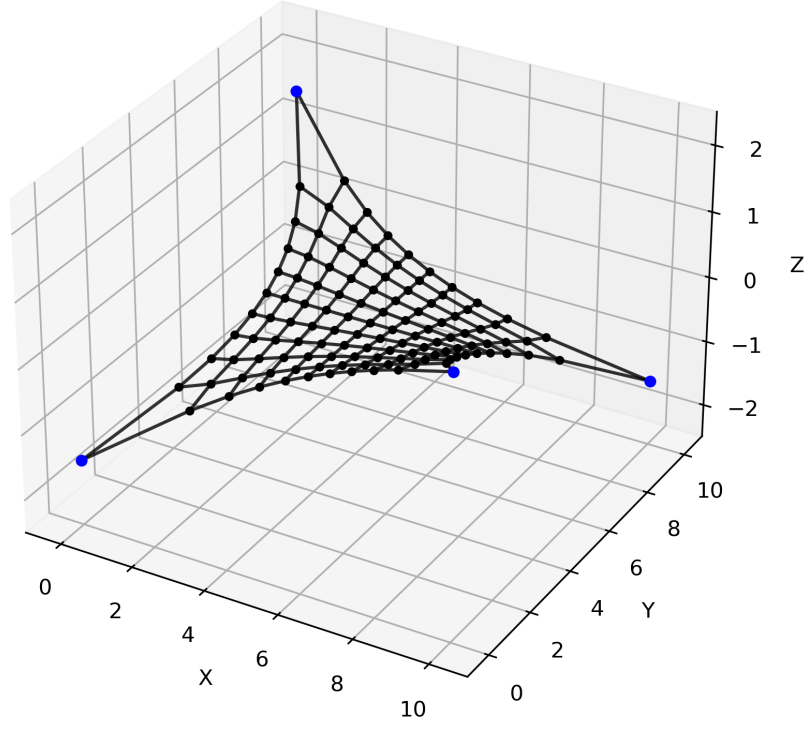
Pauletti Validation Structure

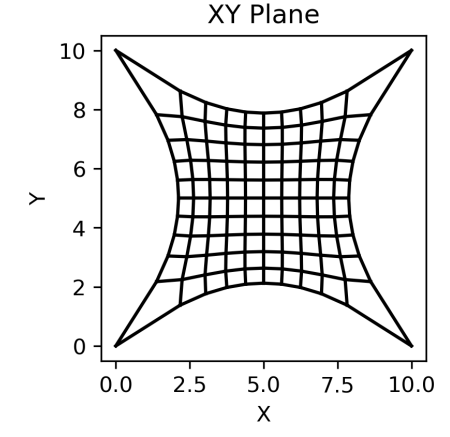
For this example, the structure is initialized with a **5:1 preload ratio** between the outer and inner members.

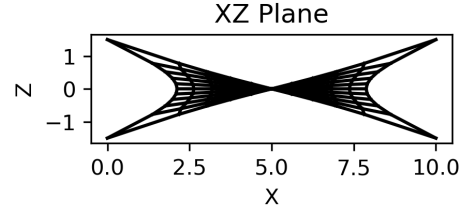
We do **not** impose any additional geometric or force constraints.

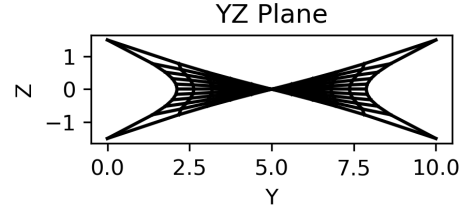
The goal is simply to let the method run and observe the equilibrium shape that emerges from this initial force distribution.

Final Structure

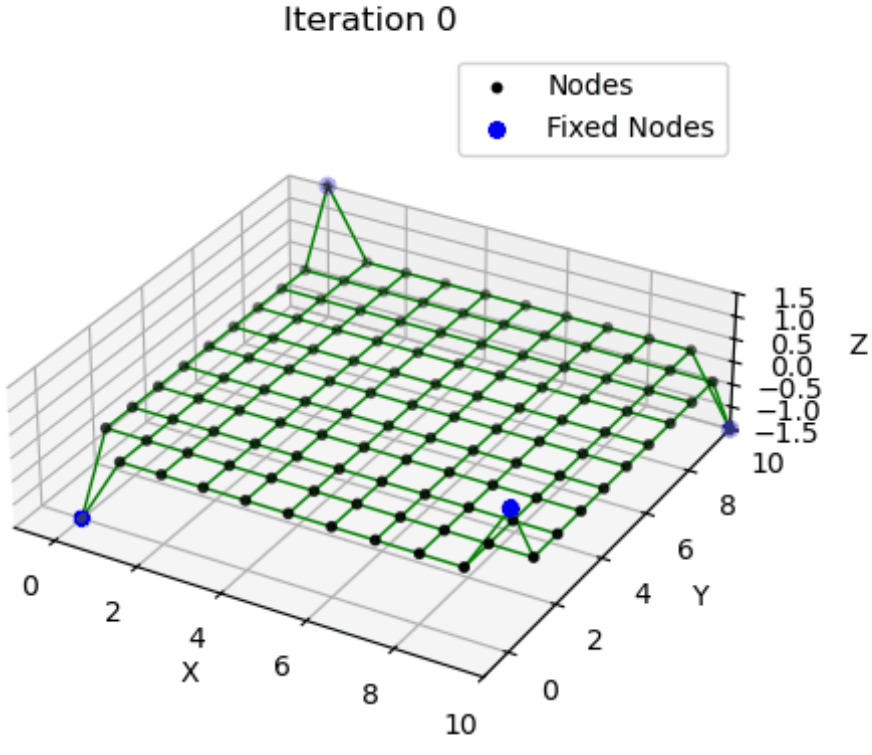




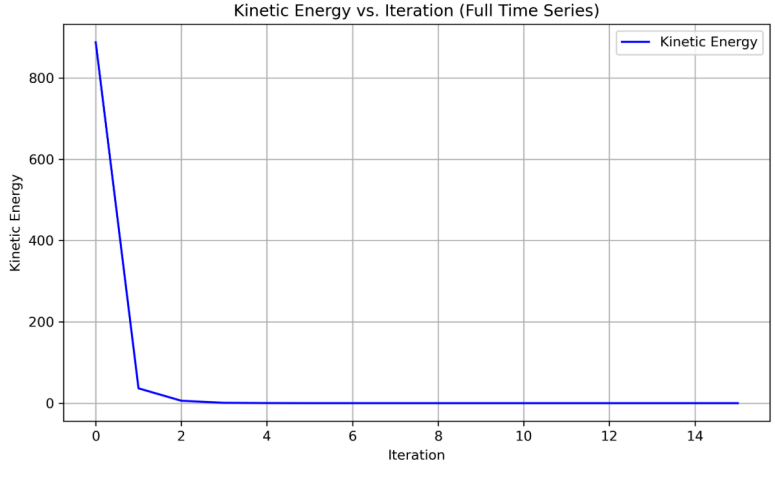




Animation (Only Visible in Jupyter Notebook)



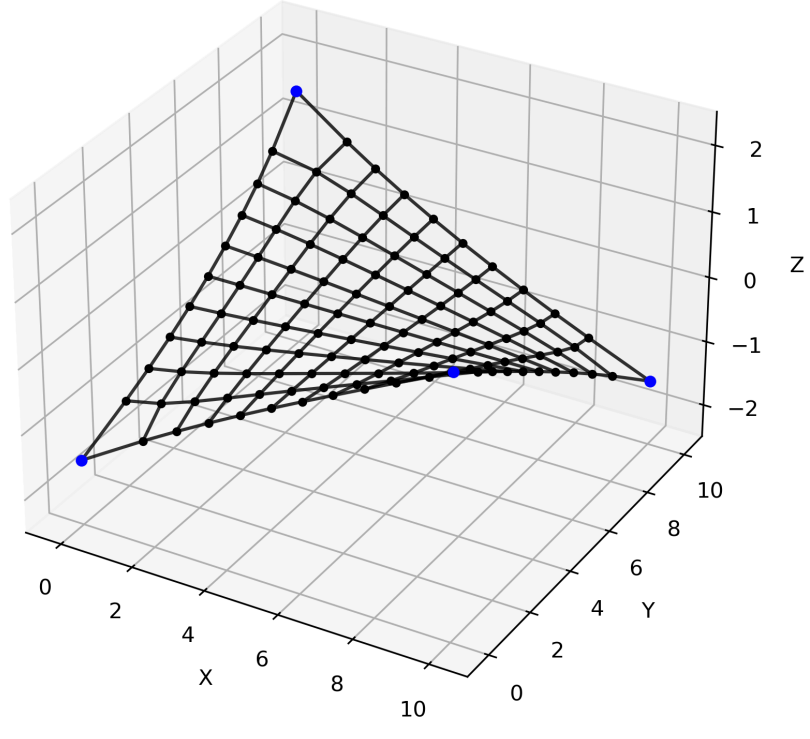
Kinetic Energy Measured Through Iterations

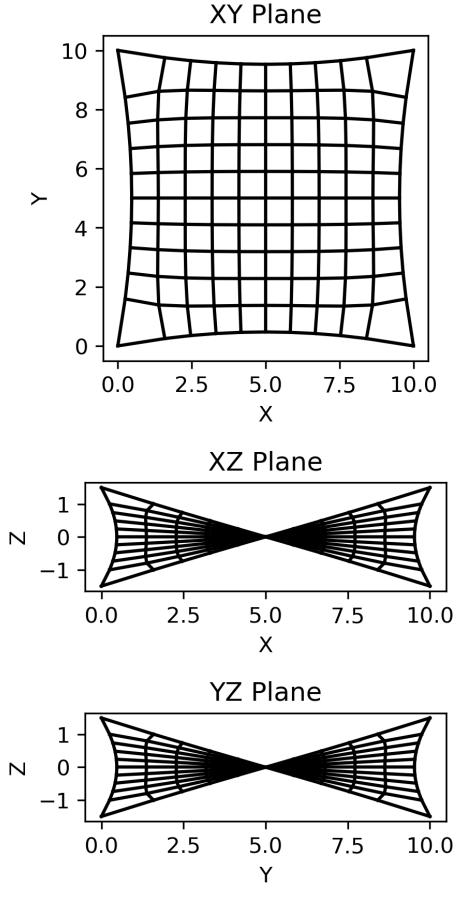



```
Iteration 000: Total Len = 147.199, Max error = 9.080e+01
Iteration 001: Total Len = 150.568, Max error = 3.369e+00
Iteration 002: Total Len = 150.298, Max error = 2.700e-01
Iteration 003: Total Len = 150.840, Max error = 5.419e-01
Iteration 004: Total Len = 151.073, Max error = 2.337e-01
Iteration 005: Total Len = 151.268, Max error = 1.949e-01
Iteration 006: Total Len = 151.370, Max error = 1.014e-01
Iteration 007: Total Len = 151.435, Max error = 6.489e-02
Iteration 008: Total Len = 151.470, Max error = 3.589e-02
Iteration 009: Total Len = 151.492, Max error = 2.128e-02
Iteration 010: Total Len = 151.504, Max error = 1.202e-02
Iteration 011: Total Len = 151.511, Max error = 6.942e-03
Iteration 012: Total Len = 151.515, Max error = 3.946e-03
Iteration 013: Total Len = 151.517, Max error = 2.256e-03
Iteration 014: Total Len = 151.518, Max error = 1.283e-03
Iteration 015: Total Len = 151.519, Max error = 7.302e-04
Convergence achieved! Performing one final update.
Final update complete. Exiting loop.
```

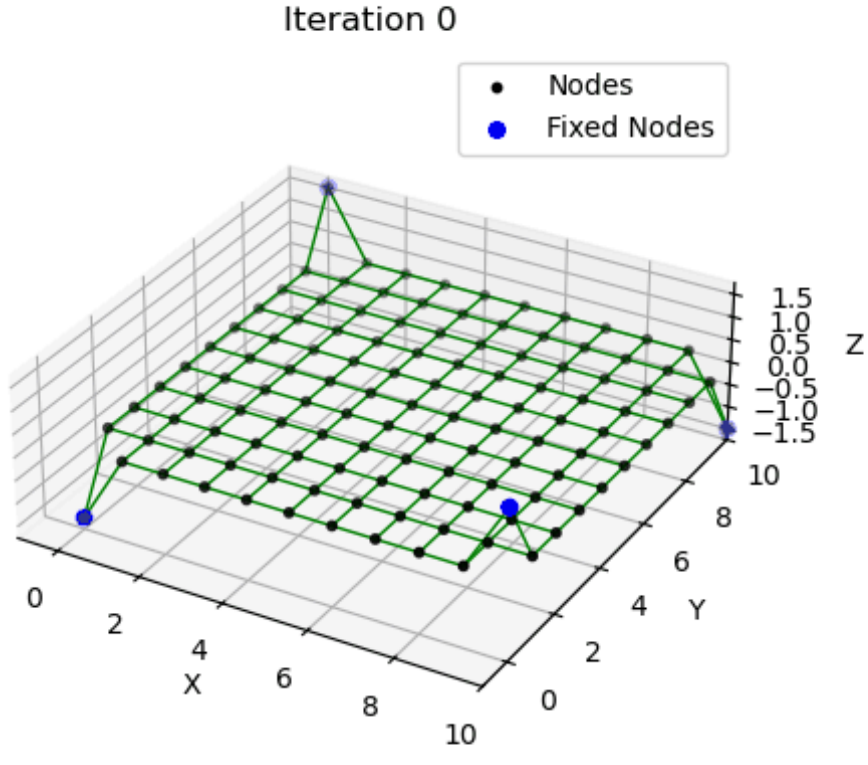
Pauletti Validation Structure, 30:1 preload ratio

Final Structure





Animation (Only Visible in Jupyter Notebook)



Testing Computational Efficiency

One of the main motivations for **Dynamic Relaxation (DR)** is computational efficiency.

Because DR uses a **lumped nodal stiffness** formulation, the system is much simpler to evaluate than the fully coupled **Stiffness Matrix (SM)** formulation.

To compare the two methods, the Pauletti validation structure was increased to a **25 × 25 grid**, with equal inner and outer force ratio:

```
def generate_struct(N=25, spacing=1.0, ratio_outer_to_inner=1):
```

Results:

- **SM**: 15 iterations, 12.85 s total time
- **DR**: 6 iterations, 1.91 s total time

Wrap-Up

In this lecture, we:

- connected the force density method to the unified equilibrium update formulation
- interpreted force density as a geometric-only, lumped stiffness model
- showed that prescribing $\mathbf{q}$ leads to a linear, one-step solution
- introduced constraints, making force densities unknown and the problem nonlinear
- used linearization (Jacobian) to solve for updates in $\mathbf{q}$
- implemented a simple iterative scheme using $\mathbf{Q} = \mathbf{L}^{-1}\mathbf{F}$
- compared FD with SM and DR methods in terms of formulation and efficiency

Main takeaway:
Force density is powerful because it simplifies the problem, when you add constraints, you move into nonlinear, iterative form finding where SM and DR methods are other methods you can use.